P2 · Algoritmos codiciosos

Conceptos previos + explicación de la práctica

Qué técnica usar, qué implementar y qué analizar

Curso 2026-2027

1. Problemas de optimización

Buscar una solución válida y, entre todas, la mejor.

Conjunto de soluciones

Todas las configuraciones que pueden construirse.

Restricciones

Condiciones que distinguen una solución válida de una inválida.

Función objetivo

Medida que se desea minimizar o maximizar.

Ejemplos

Cambio de monedas

minimizar monedas

Actividades

maximizar cantidad

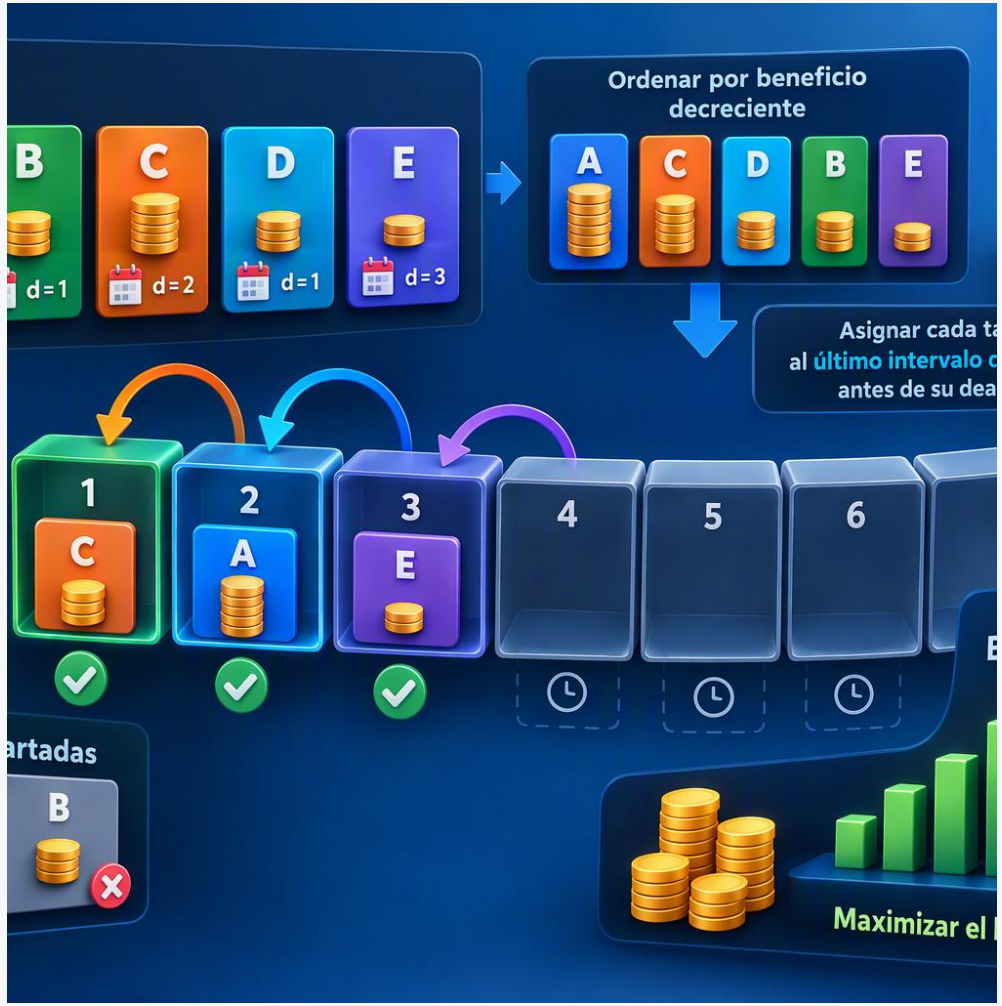
Tareas

maximizar beneficio

Una solución puede ser factible sin ser óptima.

2. Algoritmo codicioso: definición

Construye la solución mediante decisiones localmente favorables.



Decisión local
Selecciona el candidato más prometedor según una regla.

Incremental
Añade candidatos uno a uno a una solución parcial.

Factibilidad
Solo incorpora elecciones que mantienen las restricciones.

Irrevocable
No deshace ni reconsidera las elecciones anteriores.

Eficiencia
Evita explorar todo el espacio de soluciones.

“Localmente mejor” no significa automáticamente “globalmente óptimo”.

3. Anatomía de un algoritmo codicioso

Elementos que deben identificarse explícitamente.

Candidatos

Elementos disponibles para ampliar la solución.

Criterio de selección

Determina cuál se examina o selecciona primero.

Criterio de factibilidad

Comprueba si puede añadirse sin violar restricciones.

Función objetivo

Mide la calidad de la solución construida.

Condición de finalización

Indica cuándo la solución está completa.

Solución parcial

Representa lo seleccionado hasta ese momento.

La corrección depende de la regla de selección, no solo de implementar bien el recorrido.

4. Esquema general de un codicioso

Patrón de diseño reutilizable.



La elección se acepta de forma definitiva: no hay backtracking.

5. Factible, óptimo y contraejemplo

Tres ideas esenciales para analizar una estrategia.

Factible

Cumple todas las restricciones.

Óptimo

Es factible y tiene el mejor valor objetivo posible.

Contraejemplo

Demuestra que una regla no funciona para todos los casos.

Monedas [4, 3, 1] · cantidad 6

Greedy: [4,1,1]

factible, 3 monedas

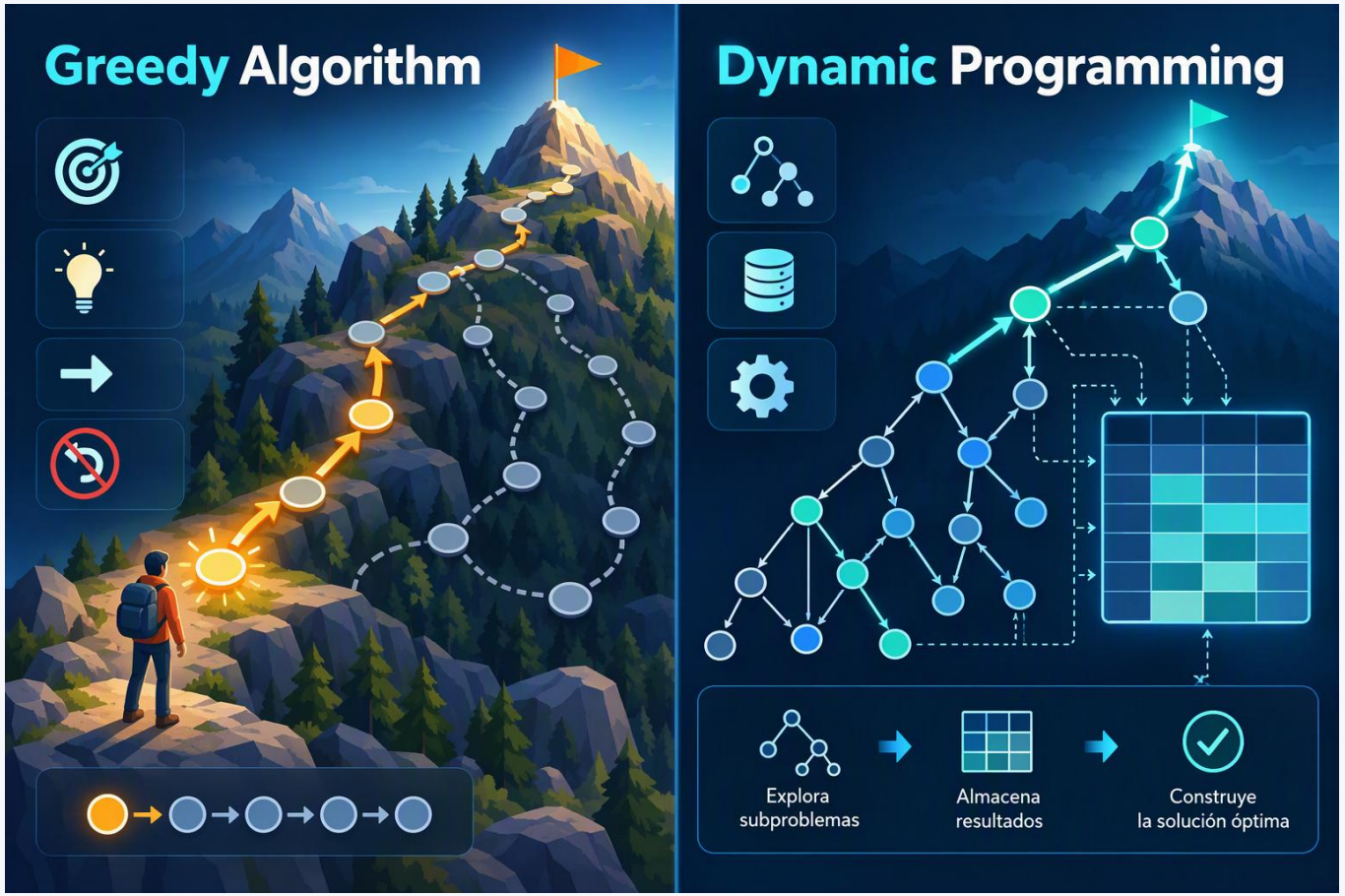
Óptimo: [3,3]

factible, 2 monedas

La regla “elegir la mayor moneda” no garantiza el óptimo en todos los sistemas.

6. Programación dinámica: definición

Resolver problemas mediante subproblemas y reutilización.



Subproblemas solapados

El mismo estado aparece repetidamente en distintas ramas.

Subestructura óptima

La solución óptima se forma con soluciones óptimas de estados menores.

Almacenamiento

Cada estado se calcula una sola vez y luego se consulta.

7. Cómo diseñar una solución dinámica

Definir la tabla antes de codificar.

1 · Estado

Información mínima que identifica un subproblema.

2 · Valor

Qué significa exactamente dp[i].

3 · Casos base

Estados cuya solución se conoce directamente.

4 · Transición

Relación entre un estado y estados menores.

5 · Orden

Secuencia que garantiza dependencias resueltas.

6 · Reconstrucción

Información para recuperar la solución elegida.

Una recurrencia puede calcular el valor óptimo, pero la reconstrucción requiere almacenar también las decisiones.

8. Ejemplo dinámico: cambio de monedas

Estado y transición de `optimal_change`.

Definición del estado — dp[x] = mejor combinación para obtener exactamente x.

Caso base — dp[0] = []

Transición — Para cada moneda c ≤ x: si dp[x−c] existe, crear dp[x−c] + [c] y conservar la candidata con menos monedas.

```
1 dp[0] = []
2 for x in range(1, amount + 1):
3     dp[x] = mejor(dp[x-c] + [c] para cada moneda válida c)
```

Tiempo: O(amount × m) · Espacio: O(amount), siendo m el número de denominaciones.

9. Memoización y tabulación

Dos maneras de evaluar la misma recurrencia.

Memoización · top-down

Se parte del problema completo. Una función recursiva solicita subproblemas y guarda sus resultados en una caché. Solo calcula los estados visitados.

Tabulación · bottom-up

Se parte de los casos base. Un bucle completa la tabla en un orden que garantiza que todas las dependencias estén disponibles.

Comparten

- Estado.
- Casos base.
- Transición.
- Resultado óptimo.

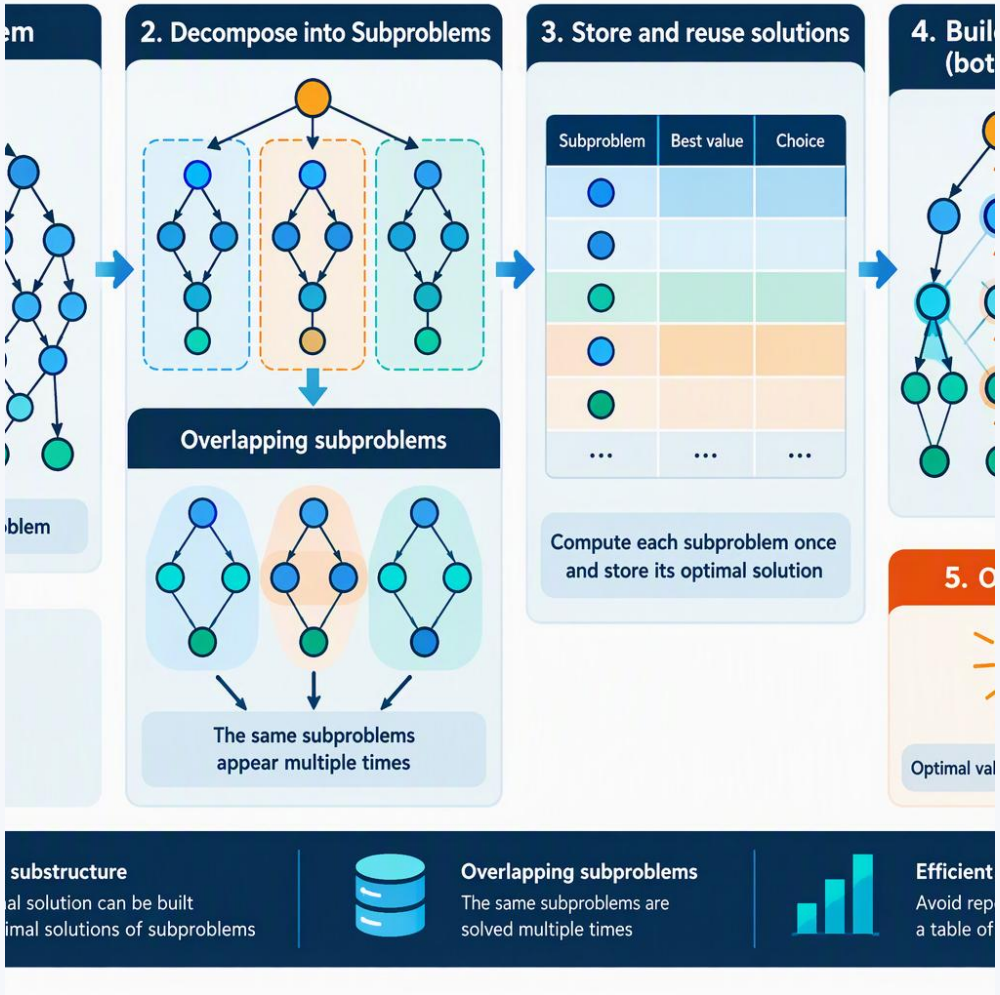
Se diferencian

- Orden de evaluación.
- Uso de recursión.
- Estados calculados.

Para `optimal_change`, la tabulación es directa: cantidades 0, 1, 2, …, amount.

10. Codicioso frente a programación dinámica

Cuándo elegir cada enfoque.



Aspecto	Codicioso	Prog. dinámica
Decisión	Local e inmediata	Compara alternativas
Compromiso	Irrevocable	Combina estados
Memoria	Generalmente baja	Tabla o caché
Optimalidad	Requiere prueba	Recurrencia correcta
Coste	Normalmente menor	Normalmente mayor

Regla práctica: usar greedy solo si puede justificarse; usar DP cuando hay alternativas solapadas y se necesita garantizar el óptimo.

11. Síntesis final

Idea esencial

Codicioso elige una opción y continúa.

Programación dinámica compara y reutiliza soluciones de subproblemas.

La corrección no depende de la intuición, sino de las propiedades del problema.

12. Estructura y entregables de la P2

Tres problemas y un análisis experimental

Cambio de monedas

Greedy + solución óptima con programación dinámica.

Selección de actividades

Máximo número de actividades compatibles.

Planificación de tareas

Máximo beneficio antes de los deadlines.

Entregables

- p2.py: implementaciones y funciones auxiliares.
- memoria.pdf: experimentos, gráficas, cuestiones y declaración de IA.
- Las firmas y estructuras de retorno deben coincidir exactamente.

Distribución de la nota

3,5 Cambio de monedas
2,5 Selección de actividades
3,0 Planificación de tareas
0,5 Cuestiones generales
0,5 Calidad global

13. Cambio de monedas: funciones a implementar

Comparar eficiencia y optimilidad.



`greedy_change` · 0,65

Mayor moneda que no supera el resto. Devuelve (selected, remainder).

`optimal_change` · 0,90

Mínimo número de monedas mediante programación dinámica.

`compare_change` · 0,45

Clasifica: optimal, suboptimal, greedy_failed e impossible.

Análisis y cuestiones: 1,50 puntos

14. Ejemplo de cambio de monedas

Una solución exacta puede ser subóptima.

Entrada

```
1 coins = [4, 3, 1]
2 amount = 6
```

Greedy

[4, 1, 1] → 3 monedas

Programación dinámica

[3, 3] → 2 monedas

Conclusión Greedy alcanza la cantidad, pero no utiliza el número mínimo de monedas. Este caso se clasifica como suboptimal.

En la memoria: analizar las cantidades 1…500 para los cuatro sistemas monetarios y calcular porcentajes por categoría.

15. Selección de actividades

Elegir la actividad que termine antes.



`activity_selection` · 1,00

Ordenar por end, después start e identifier.

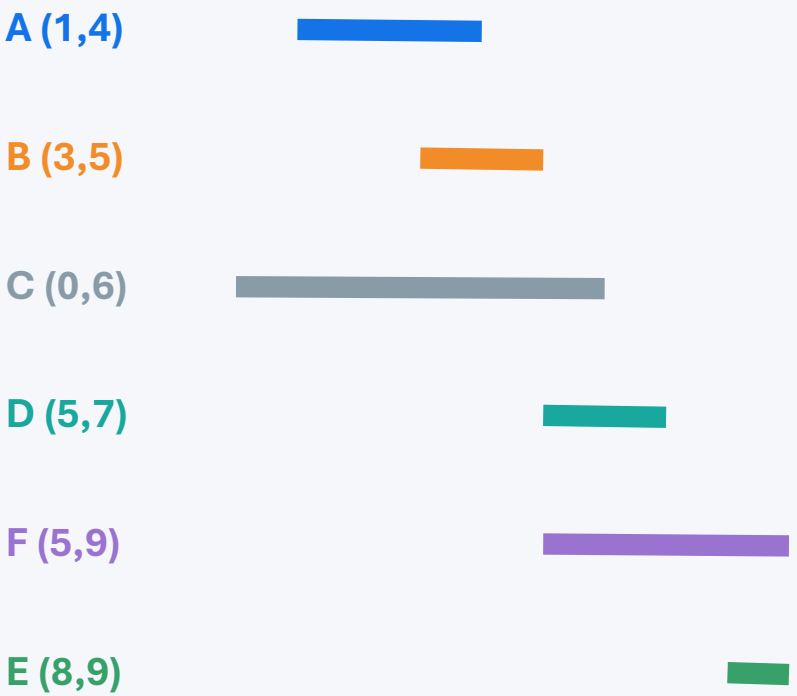
- Seleccionar la primera actividad.
- Aceptar otra si start ≥ último end.
- Devolver una lista de cardinalidad máxima.
- No modificar la lista original.

`generate_activities` · 0,35

start 0…10000 · duración 1…100 · IDs A0…

16. Ejemplo de selección de actividades

Ordenar primero; seleccionar después.



A (1, 4)
Primera que termina

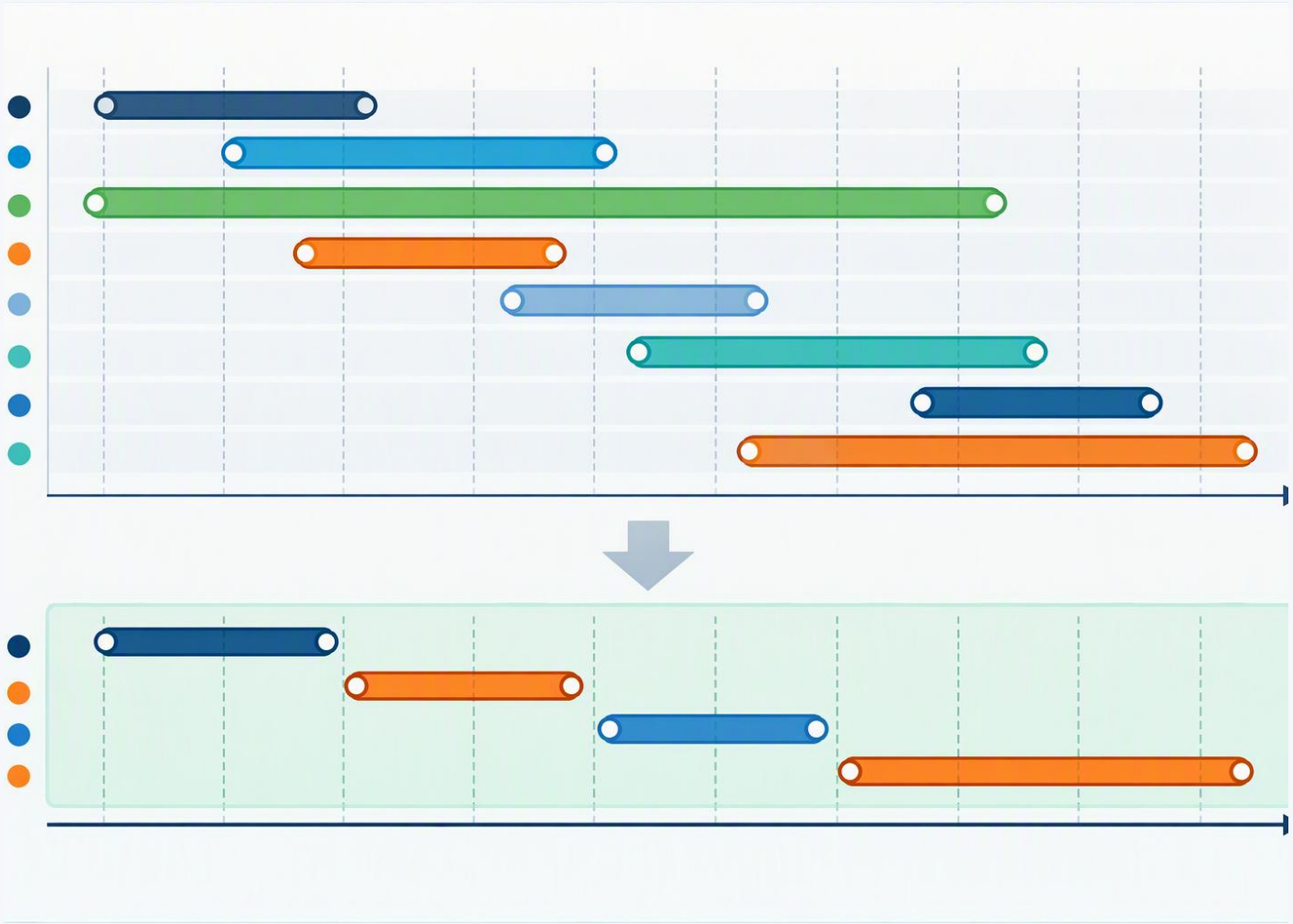
D (5, 7)
Compatible con la anterior

E (8, 9)
Compatible con la anterior

Resultado: 3 actividades compatibles

17. Planificación de tareas con deadline

Maximizar el beneficio total.



`job_scheduling` · 1,25

Ordenar por profit decreciente y asignar al último slot libre ≤ deadline.

- Duración de cada tarea: 1 unidad.
- Huecos vacíos: None.
- Longitud de schedule: max(deadline).
- Retorno: (schedule, total_profit).

`generate_jobs` · 0,35

profit 1…1000 · deadline 1…max(1,n//10).

18. Ejemplo de planificación

Colocar cada tarea lo más tarde posible.

Orden por beneficio

A	C	D	B	E
p=100 d=2	p=27 d=2	p=25 d=1	p=19 d=1	p=15 d=3

schedule

1	2	3
C	A	E

Salida esperada

```
1 ([('C',27,2), ('A',100,2), ('E',15,3)], 142)
```

Asignar tarde preserva los huecos tempranos para tareas con deadlines más restrictivos.

19. Diseño experimental común

Qué medir y cómo hacerlo de forma reproducible.

Repeticiones

Al menos 10 mediciones temporales por configuración.

Semillas

Mismas entradas para comparar algoritmos o tamaños.

Tiempo medido

Incluir la ordenación; excluir generación de datos y gráficas.

Cada gráfica debe incluir

- Título descriptivo.
- Ejes y unidades.
- Leyenda si hay varias series.
- Escala adecuada.

La memoria debe interpretar

- Tendencias observadas.
- Relación con la complejidad.
- Anomalías o resultados inesperados.
- Compromiso eficiencia–optimalidad.

20. ¿Qué debe contener memoria.pdf?

Evidencias del trabajo realizado.

- Metodología experimental: entorno, semillas y repeticiones.
- Tablas y gráficas solicitadas en cada bloque.
- Respuestas razonadas a las cuestiones.
- Análisis de complejidad temporal y espacial.
- Interpretación de resultados, no solo capturas.
- Declaración de uso de IA generativa.

Qué distingue una buena memoria

- Coherencia con el código entregado.
- Conclusiones respaldadas por datos.
- Complejidades justificadas.
- Gráficas legibles y comparables.
- Uso preciso de los conceptos: factibilidad, criterio codicioso y optimalidad.

No es necesario copiar listados completos de código.

21. Checklist antes de entregar

Comprobaciones finales.

- ✓ p2.py se importa sin ejecutar pruebas.
- ✓ Se han probado los casos límite.
- ✓ Las firmas coinciden exactamente.
- ✓ memoria.pdf incluye todas las gráficas.
- ✓ No quedan print ni input.
- ✓ Las cuestiones están razonadas.
- ✓ Los retornos tienen el formato solicitado.
- ✓ Los ficheros tienen el nombre correcto.

Objetivo: implementar correctamente y demostrar que entendéis por qué funciona cada estrategia.